

NYPC 2026 Master Qualification Round Replay

On a Deep Reinforcement Learning System for Efficiently Training Transformer Models to Master the "NEXT NATION" Game

팀명: 안녕하세요민이입니다잘부탁드립니다

사용한 LLM의 제공자(플랜): Claude Pro

사용한 LLM 모델: Claude Sonnet 5, Opus 4.8, Fable 5

문제 풀이

1. 초기 휴리스틱 AI (2026.06.29.)

우리 팀이 처음부터 강화학습 방향으로 시작한 것은 아니었다. 나중에 강화학습으로 AI 모델을 학습시킨다고 해도, 결국 효과적인 observation space나 action space를 설계하기 위해서는 해당 게임에 대한 전반적인 지식이 필요하다고 생각되어, 어떤 게임인지 파악도 할 겸 휴리스틱 AI 플레이어를 먼저 코딩해보기로 하였다.

이 "NEXT NATION"이라는 게임 규칙을 보고 우리 팀이 처음 했던 관찰은 “상대보다 적은 수의 전사들을 격전지에 밀어넣는 것은 무조건 손해”라는 점이였다. 이 게임의 전투 시스템을 보면, 예를 들어 5명의 전사들과 3명의 전사들이 맞부딪혔을 때 5명의 전사들은 3번의 공격을 받을 동안 3명의 전사들은 5번의 공격을 받는다. 이는 결국 전투가 벌어졌을 때, 전사 수가 더 많은 쪽의 팀이 훨씬 적은 양의 희생으로 상대방의 전사들을 물리칠 수 있음을 의미한다.

따라서 우리 팀은, 우선 초반에는 게임판의 원점을 기준으로 해서 아군 본부 쪽에 있는 거점들에 기지를 먼저 건설해 나가면서 확장을 하고, 상대방의 기지를 공격할 때엔 공격하는 아군 전사 수가 상대방의 기지에 주둔 중인 상대방 전사 수보다 많을 때만 공격하도록 했다. 이 휴리스틱 AI로 06.29. 15:00:00에 있었던 중간 평가에서 8등을 하였다. 그러나 이날 팀 회의에서는, 게임 규칙이 복잡한 편이고, 플레이를 하다 보면 굉장히 다양한 상황이 연출될 수 있어 그러한 상황들을 모두 커버하는 휴리스틱 AI 플레이어를 코딩하기에는 현실적으로 힘들 수 있겠다는 의견이 나왔다. 이에 팀원 한 명은 강화학습 AI 플레이어를 설계하고, 팀원 한 명은 계속 휴리스틱 AI 플레이어를 다듬는 것으로 역할 분담을 하였다.

2. 강화학습 AI 1 (2026.06.30.~2026.07.04.)

(1) 사용한 강화학습 알고리즘과 Reward System 설계

우리 팀에서 사용한 강화학습 알고리즘은 PPO(Proximal Policy Optimization)로, 뛰어난 안정성 덕분에 보통 강화학습 실험 시에 baseline으로 가장 먼저 시도되는 알고리즘이다.

이 알고리즘에 대해 간략히 소개하자면, 현재 state에 대한 정보가 입력되었을 때 가능한 action들 각각의 확률을 출력하는 actor net 모델과, 현재 state가 agent의 입장에서 얼마나 좋은지(state value)를 출력하는 critic net 모델 이 2개의 모델을 학습시키는 방식이다. 먼저 critic net의 경우에는 게임 플레이 데이터를 받아 특정 state 이후에 reward를 받았는지를 확인해 그 state가 agent 입장에서 얼마나 좋은지를 학습시킨다. 그리고 actor net 이 출력한 action이 만들어낸 state에 대해 critic net이 평가를 함으로써 그 state가 좋으면 해당 action의 확률을 높이고, 나빴다면 해당 action의 확률을 낮추는 식으로 actor net을 학습시킨다. 이렇게 ‘플레이 데이터 수집 -> actor net, critic net 학습’ 이 사이클을 1 iteration이라고 하는데, 보통 수백~수천 iteration을 반복해 actor net이 점차 최적의 정책으로 수렴하도록 만든다. 이때 한 iteration에서 actor net을 학습시킬 때 각 action들의 확률이 크게 변하지 않도록 제한하여, action 확률이 갑자기 너무 크게 변동해 발생하는 성능붕괴 문제를 완화시킨다.

PPO의 단점으로는 비교적 낮은 데이터 효율성을 꼽을 수 있다(actor net과 critic net 학습 시에는 바로 그 iteration에서 수집한 데이터만 사용할 수 있고, 이전 iteration들에서 수집한 데이터는 사용이 불가능하다). 즉, 어느 정도 성능이 나오기 위해서는 다른 강화학습 알고리즘들에 비해 agent가 게임을 훨씬 많이 플레이해봐야 한다는 것이다. 하지만 "NEXT NATION" 게임 특성상 게임을 플레이하는 전략은 복잡할지 몰라도, 게임 규칙 자체는 컴퓨터 입장에서 굉장히 계산하기 쉬운 환경이었다. 따라서 PPO 알고리즘으로 일주일 내에 충분히 좋은 성능의 agnet를 학습시킬 수 있으리라 판단하였다. 추가로 claude code를 이용해, CUDA로 많은 게임들을 동시에 병렬적으로 돌리는 시스템을 구성하여 게임 플레이 데이터 축적을 가속화시켰다. 이후 학습을 돌려보니 실제로 데스크탑(부록 참고) 하나 기준으로 약 45시간 정도면 매우 좋은 성능을 내는 agent를 학습시킬 수 있었다.

보통 학습 데이터를 많이 얻기 힘든 상황이거나 agent가 reward를 얻기 매우 힘든 상황인 경우엔 보조 reward 항을 넣어줌으로써 어떤 action이 좋은 action인지를 더 빠르게 알려준다. 그러나 이 "NEXT NATION" 게임은 게임이 종료됐을 때 승(+10), 무(0), 패(-10) reward만 넣으면 충분했다. 아마 승리 조건이 그렇게 까다롭지 않으면서 max step 수도 200으로 작은 편이라 그랬던 것으로 생각한다.

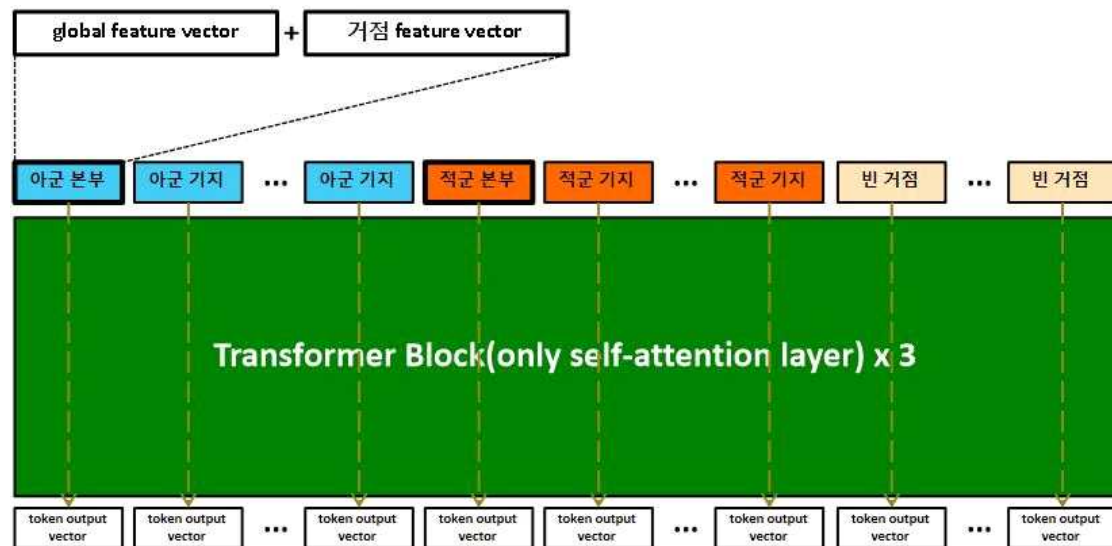
(2) 모델 구조

actor net이나 critic net의 전체적인 모델 구조는 self attention layer로만 구성된 transformer 모델로 결정하였다. 단순 MLP만으로 게임의 정보들을 효과적으로 소화하는 모델을 만들려면 크기가 엄청 커져야해서 효율적인 학습이 불가능할 것 같았고, CNN도 이 게임의 특성상 actor net이 ‘state->최적 action’ 매핑을 학습하기 어려워보였다.

우리 팀이 학습 모델의 구조를 설계할 때 이 게임에서 주목한 점은, 중요한 정보들이 거점들에 몰려있다는 것과, 각 거점들의 정보를 neural network가 받을 때 거점 순서가 전혀 중요하지 않다는 점이다(예를 들어 모든 거점들의 정보를 1차원 텐서로 concat해서 critic net에 넣는다고 할 때, 아군 본부의 정보 벡터 다음에 적군 본부의 정보 벡터를 concat해서 넣었을 때와 적군 본부의 정보 벡터에 아군 본부의 정보 벡터를 concat해서 넣었을 때 critic net의 출력값에 차이가 있으면 안 된다). 이렇게 node index permutation에 대해 equivariant함을 만족하는 모델의 구조에는 self attention layer로 구성된 transformer 모델과 GCN(graph convolutional network)이 있는데, 게임 정보의 복잡성을 고려했을 때

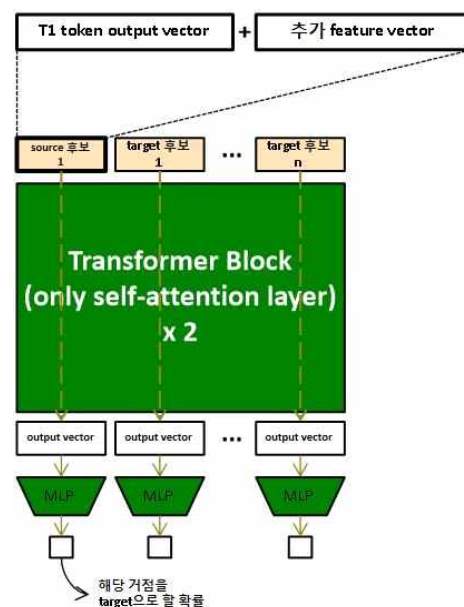
transformer 모델이 더 좋은 성능을 보일 것 같아 우리 팀은 transformer 모델로 결정했다.

actor net은 2개의 transformer 모델로 이루어져 있고, critic net은 1개의 transformer 모델로 이루어져 있다. actor net이 2개의 transformer 모델로 이루어져 있는 이유는 이 게임의 action 중 이동 명령의 경우 어디(source)에 있는 전사를 어디(target)로 보낼 건지를 정해야 하기 때문에, T1 transformer에서 source 거점들을 정하고, T2 transformer에서 각 source 거점들에 대해 target 거점을 정한다고 보면 된다. critic net의 transformer 모델은 T1과 동일하며, T1의 구조는 아래와 같다.



global feature와 거점 feature(observation space 참고)가 concat되어 하나의 token vector를 이루고, 그 token vector들이 T1으로 들어가는 형태다(global feature 부분은 모든 token vector들이 동일함). 그리고 token output vector들이 나오면, actor net의 경우엔 건설 명령, 이동 명령, 훈련 명령 여부를 결정하는 데에 사용되고(action space 참고), critic net의 경우엔 각 token output vector를 작은 MLP에 통과시켜 나온 value값의 평균으로 해당 state의 value를 예측한다.

T2의 구조는 우측과 같다. T1에서 나온 token output vector와 추가 feature vector(observation space 참고)가 concat되어 token vector를 이루고, 그 token vector들이 T2로 들어가는 형태다. T2는 actor net에서 각 source 거점에 대해 target 거점을 결정하는 모델이므로, T2에는 항상 하나의 source token vector와 여러 개의 target 후보(사실상 모든 거점들이다) token vector들이 들어간다. 그리고 출력된 output vector를 간단한 MLP에 넣어 scalar값이 나오게 한 뒤, 이 scalar값들을 모두 stack해 1차원 텐서로 만들고 softmax를 통과시켜 해당 target 후보를 정말 target으로 결정할 확률을 출력하게 된다. 만약 target으로 source 거점 자기



자신이 뽑힌다면 그 source 거점에 대해서는 이동 명령을 내리지 않게 된다. 한 state에 대해 action을 결정할 때, actor net에서 T1은 한 번만 추론하면 되지만, 이동 명령의 source로 여러 거점이 가능한 경우 T2는 source로 가능한 거점 개수만큼 추론해야 한다. 이 게임에서는 특정 거점에 있는 전사들 중 원하는 전사들만 지정해서 이동시키는 것이 가능하지만, 문제 단순화를 위해 특정 거점에 이동명령을 내리면 그 거점에 있는 노동 및 이동 중이 아닌 전사들은 모두 이동하도록 하였다(action space 참고).

(3) Observation Space

모델이 받는 정보는 결국 global feature vector, 거점 feature vector, 그리고 T2에서 사용되는 추가 feature vector 이렇게 3가지로 요약할 수 있다. actor net과 critic net에서 이 3가지는 동일하게 했다.

Global Feature Vector 구성			
1	현재 턴 수	$x/10-10$	1 dim
2	각 팀의 현재 전사 총 수	$\ln(x/10+1)$	2 dim
3	각 팀의 본부 레벨 수	$\ln(x+1)$	2 dim
4	각 팀의 골드 수치	$\ln(x/100+1)$	2 dim
5	이전 턴에 각 팀이 벌어들인 골드 수	$\ln(x/10+1)$	2 dim
6	각 팀의 본부 & 기지의 레벨 수 총합	$\ln(x/5+1)$	2 dim

사실 실제 게임 환경에서는 정확한 적군의 골드 수치를 알 수 없다. 그렇지만 일단 학습 환경에서는 정확한 적군의 골드 수치가 actor net과 critic net 둘 모두에 들어가게 했고, 제출용 agent 코드를 짤 때는 얻을 수 있는 정보들을 토대로 최대한 상대방의 골드 수치를 예측하게 했다.

거점 Feature Vector 구성			
1	해당 거점에 존재하는 아군 전사 수	$\ln(x+1)$	1 dim
2	해당 거점에 존재하는 적군 전사 수	$\ln(x+1)$	1 dim
3	해당 거점에 존재하는 아군 기지 레벨 수	$\ln(x+1)$	1 dim
4	해당 거점에 존재하는 적군 기지 레벨 수	$\ln(x+1)$	1 dim
5	해당 거점에 존재하는 아군 본부 레벨 수	$\ln(x+1)$	1 dim
6	해당 거점에 존재하는 적군 기지 레벨 수	$\ln(x+1)$	1 dim
7	해당 거점의 아군 기지/본부의 포탑 공격력	$\ln(x+1)$	1 dim
8	해당 거점의 적군 기지/본부의 포탑 공격력	$\ln(x+1)$	1 dim
9	해당 거점의 아군 기지/본부의 노동 가능 전사 수	$\ln(x+1)$	1 dim
10	해당 거점의 적군 기지/본부의 노동 가능 전사 수	$\ln(x+1)$	1 dim
11	해당 거점의 아군 기지/본부의 현재 체력 수치	$\ln(x+1)$	1 dim
12	해당 거점의 적군 기지/본부의 현재 체력 수치	$\ln(x+1)$	1 dim
13	해당 거점에 존재하는 이동 중, 노동 중이 아닌 전사 수	$\ln(x+1)$	1 dim
14	해당 거점에 존재하는 이동 중이 아닌 아군 전사들의 체력 총합	$\ln(x+1)$	1 dim
15	해당 거점에 1턴, 2턴, ..., 5턴 뒤에 도착할 아군 전사 수 총합	$\ln(x+1)$	5 dim
16	해당 거점에 1턴, 2턴, ..., 5턴 내에 도달할 수 있는 적군 전사 수 총합	$\ln(x+1)$	5 dim
17	바로 이전 턴과 비교했을 때의 16번째 feature의 변화량	$\ln(x+1)-\ln(x+1)$	5 dim
18	해당 거점의 x, y좌표(모든 거점들의 x, y좌표 값들 중 최댓값을 10으로, 최솟값을 -10으로 정규화)	$[-10, 10]$	2 dim

여기서 ‘해당 거점에 1턴, 2턴, ..., 5턴 내에 도달할 수 있는 적군 전사 수 총합 feature’의 경우엔, 아군과 달리 적군의 전사들은 어디로 이동 명령을 받았을지 알 수 없기 때문에 ‘도달할 수 있는’ 적군의 수치를 계산하도록 했다. 그리고 ‘해당 거점에 1턴, 2턴, ..., 5턴

내에 도달할 수 있는 적군 전사 수 총합 feature'의 변화량 자체를 feature로 넣어준 이유는, 모델 구조 상 적 전사들의 정확한 현재 위치들과 이동방향을 feature로 넣어주는 것은 불가능했기 때문에 그 대신 각 거점들에 적 전사들의 공격이 얼마나 임박했는지를 의미할 수 있는 feature라도 필요하다고 판단했기 때문이다.

추가 Feature Vector 구성			
1	해당 거점에 존재하는 적군 전사 수 + 해당 거점에 있는 적 본부/기지의 포탑 공격력	$\ln(x+1)$	1 dim
2	해당 거점에 존재하는 적군 전사들의 체력 수치 총합 + 해당 거점에 있는 적 본부/기지의 체력 수치	$\ln(x/5+1)$	1 dim
3	해당 거점에 존재하는 아군 전사 수 + 해당 거점에 있는 아군 본부/기지의 포탑 공격력	$\ln(x+1)$	1 dim
4	해당 거점에 존재하는 아군 전사들의 체력 수치 총합 + 해당 거점에 있는 아군 본부/기지의 체력 수치	$\ln(x/5+1)$	1 dim
5	source 거점에 존재하는 이동 중도, 노동 중도 아닌 아군 전사 수	$\ln(x+1)$	1 dim
6	source 거점에서 해당 거점까지 전사가 이동하는 데에 걸리는 턴 수	$\ln(x+1)$	1 dim

feature를 구성할 때 A거점에서 B거점으로 전사가 이동하는 데에 걸리는 턴 수를 계산해야 하는 경우가 많은 것을 볼 수 있다. 효율성을 위해, 게임 처음에 각 구역들의 위치 정보가 주어졌을 때 모든 구역에서 모든 거점으로 이동하는 데 걸리는 최소 턴 수를 미리 계산해 저장해놓도록 했다.

마지막으로, 이 게임은 플레이어의 본부가 게임 판의 왼쪽과 오른쪽 둘 중 한 곳에 위치해 있게 된다. 우리 팀은 PPO 학습 시에는 항상 학습 데이터를 쌓는 actor net의 본부가 왼쪽, opponent의 본부가 오른쪽에 위치하도록 고정해놓았다. 그리고 중간 평가 제출용 agent는 아군 본부가 오른쪽에 위치하는 경우 observation space의 모든 x, y값을 원점 대칭시키도록 함으로써 actor net의 입장에서 항상 아군 본부가 왼쪽에 있는 것처럼 state를 인식하도록 했다.

(4) Action Space

(a) 건설 명령

actor net T1의 token output vector에 간단한 MLP를 붙이고, 그 MLP에서 1차원의 logit 값이 출력되게 한 뒤 sigmoid 함수를 사용해 해당 거점에 건설 명령을 내릴지 말지를 결정하게 했다. 여기서 핵심 아이디어는 actor net은 해당 거점에 기지 건설 명령을 내릴지, 업그레이드 명령을 내릴지 등 세부 사항은 결정할 필요 없이 건설 명령 여부만 결정하게 한 것이다. 만약 건설 명령을 내린 거점에 아군 본부나 기지가 없고, 적군 전사도 없다면 기지를 새로 건설하도록 했고, 만약 적군 전사가 있다면 건설 명령이 불가능하게 막았다. 건설 명령을 내린 거점이 아군 기지가 있었다면 기지 업그레이드 명령을, 아군 본부가 있었다면 본부 업그레이드 명령으로 변환해 게임 환경에 전달하였다.

학습을 진행할수록 에이전트가 기지 업그레이드를 하지 않는 경향을 보였다. 이에 대해 우리 팀에서는, 기지를 업그레이드 하더라도 지금 당장 골드 생산에 이득이 되는 것은 아니기 때문에 actor net이 자연스럽게 기지 업그레이드를 회피하게 되어 발생한다고 보았다. 따라서 특정 거점의 아군 기지를 업그레이드하는 경우, 무조건 가장 가까운 거점에 있는 이동/노동 중이 아닌 아군 전사 1명이 해당 거점으로 이동하도록 강제해 곧바로 골드 생산량이 증가하도록 해보았다. 그럼에도 불구하고 학습이 진행될수록 기지 업그레이드 action은 억

제되는 모습을 보였다. 따라서 애초에 기지 업그레이드는 게임 구조적으로 효율성이 굉장히 떨어지는 action이라는 결론이 내려졌다.

그러나 본부 업그레이드는 달랐다. 학습된 에이전트가 플레이한 게임들의 리플레이를 분석하다 보니, 게임 종료 몇 턴 전부터 수비만 적당히 하면서 본부 업그레이드에 집중했다면 충분히 이겼을 게임을 계속 상대 본부 공격에만 집중하다가 아쉽게 비기는 경우가 많이 보였다. 이는 본부 업그레이드를 하기 위해서는 다른 action들보다 훨씬 많은 골드가 필요하고, 학습 초반에는 본부 업그레이드를 하기 충분한 골드가 모일 때까지 다른 action들을 모두 억제하는 시퀀스가 나오기 어렵다보니 본부 업그레이드의 이점을 잘 학습하지 못하는 것으로 분석했다. 특히 본부를 3레벨까지는 업그레이드하는 모습을 종종 보였음에도 4레벨 업그레이드는 전혀 하지 못했다. 따라서 PPO 학습 시에 4레벨/5레벨 본부 업그레이드 action에 한해 해당 턴에 골드가 부족하더라도 업그레이드를 수행할 수 있게 해준 대신, 그 다음 턴부터 업그레이드 action 수행을 위해 충분한 골드가 모이기 전까지는 골드를 소비하는 모든 action은 아예 금지해버리는 트릭을 적용해 보았다. 그 결과 상대의 수비가 뚫기 힘든 경우 본부 업그레이드에 집중해 4레벨 업그레이드까지 함으로써 승리하는 모습을 종종 보여줬다. 그러나 본부를 5레벨까지 업그레이드하는 모습은 한 번도 보지 못했는데, 이 action도 기지 업그레이드 action처럼 애초에 게임 구조적으로 효율성이 떨어지는 action 일 것이라 결론내렸다.

(b) 이동 명령

이 게임에서는 원래 이동 중이 아닌 아군 전사 하나하나에 대해 이동 명령을 따로 전달할 수 있다. 그러나 actor net에게 각 전사들의 정보를 따로따로 전달하고, 이번 턴에 각 전사들을 이동시킬지 말지, 이동시킨다면 어디로 이동시킬 건지까지 학습시키는 것은 굉장히 비효율적이라는 생각이 들었다. 게다가 그러한 이동 명령 방식은, 거점 단위로 정보를 주는 우리의 actor net transformer 모델에도 적용하기가 까다로웠다. 따라서 우리 팀은 actor net이 이번 턴에 어떤 거점에 이동 명령을 내릴지(source), 그리고 그 이동 명령의 목적 거점은 어디로 할 것인지(target)만 정하도록 했다. actor net T1이 이번 턴의 source 거점들을 정하면, T2가 각 source 거점들에 대해 target 거점들을 정하는 구조이다. 따라서 한 턴에 여러 source에 이동 명령을 내리는 것은 가능하지만 하나의 source에는 항상 하나의 target으로 보내는 이동 명령만 내릴 수 있으며, target으로는 오로지 거점만(거점이 아닌 지역은 target으로 지정 불가능) 지정할 수 있다. source 거점에 있는, 이동 중이 아닌 전사들은 무조건 모두 그 이동 명령을 따르도록 하였고, 만약 source 거점에 아군 본부나 기지가 있다면 이동 중이 아닌 전사들 중 가장 체력 수치가 낮은 "노동 가능 전사 수"만큼만 source 거점에 남겨두었다.

이 action space의 이동 명령 단순화가 특히 "NEXT NATION" 게임에 잘 맞았던 걸로 보인다. 이 게임에서는 두 팀의 전사들이 한 지역에서 격돌하면 전사 수가 많은 팀이 무조건 이득을 볼 수 밖에 없는데, 이 action space는 이동 명령을 내릴 때 한 지역 내에 있는 전사들은 모두 한꺼번에 이동시키도록 강제하기 때문에 게임이 진행될 수록 같은 팀의 전사들이 한 지역에 모이면서 자연스럽게 점점 뭉쳐 다닐 수밖에 없다. 또한 아군의 본부나 기지에는 '노동 가능 전사수' 만큼은 무조건 남기도록 한 것도 결과적으로 봤을 때 좋은 규제였던 것 같다. 따라서 이동 명령을 이렇게 단순화시킨 것이 actor net이 매우 빠르게 좋은 policy를 학습하도록 만들었으며, 이 점이 우리 팀의 RL agent가 다른 팀의 RL agent들보다 강력했던 가장 큰 이유로 생각된다.

(c) 훈련 명령

actor net T1의 token vector output 각각에 간단한 MLP를 붙이고, 그 MLP에서 4차원의 logit값들을 출력하게 한 뒤 요소별 평균을 적용해 actor net 전체에서 4차원의 단일 벡터가 출력되도록 했다. 아군 본부를 최대로 업그레이드 하더라도 최대 훈련 가능 전사 수는 3명이기 때문에, 그 4차원의 logit 값들에 softmax 함수를 적용해 이번 턴에 전사를 0명, 1명, 2명, 3명 훈련시킬 확률을 의미하도록 하였다. 물론 아군 본부의 현재 레벨 상 2명, 3명 훈련시키는 것은 불가능하다면, 그 확률 값을 0으로 masking하였다.

(d) 골드 제한 적용

한 턴에 내릴 수 있는 명령은 건설 명령, 이동 명령, 훈련 명령이 있고, 모든 명령들은 어느 정도의 골드가 소비된다. 따라서 actor net이 결정한 명령들이 많아도 현재 갖고 있는 골드 수가 적어 실제로는 다 실행할 수 없을 수도 있다. 어차피 한 턴에 여러 명령들을 내려도 게임 시스템 상 건설 명령, 이동 명령, 훈련 명령 순서대로 수행되기 때문에, actor net이 결정한 명령들도 이 순서에 따라 골드가 부족한지 확인하면서 최종적으로 수행할 명령들을 결정하게 했다. 이때 한 턴에 여러 개의 명령이 가능한 건설 명령과 이동 명령의 경우에는 골드 여건 상 그 중 일부만 수행할 수 있을 경우, 한 개씩 샘플링하면서 가능한 만큼만 수행하도록 했다. 예를 들어 이번 턴에 actor net이 건설 명령 2개, 이동 명령 3개, 훈련 명령 1개를 결정했지만 골드 여건 상 이동 명령의 일부까지만 가능하다면, 건설 명령 2개를 모두 수행하고 이동 명령 중에 실제로 수행할 명령을 1개씩 뽑다가 골드가 부족해지면 거기까지만 수행하는 식이다.

(5) Self-Play 학습 방식과 Opponent Pool System

우리 팀의 PPO 학습은 오로지 actor net 자기 자신과의 경기만으로 학습 데이터를 수집했다. 초반에는 간단한 휴리스틱 AI를 먼저 만들어서 그 휴리스틱 AI의 경기 데이터로부터 imitation learning을 먼저 하고 self-play 학습 방식으로 넘어갈까도 고민했지만, 실제로 학습을 돌려보니 처음부터 self-play 학습 방식으로 해도 학습이 충분히 빠르게 이루어졌다. 다만 self-play만으로 학습을 하게 되면 actor net 자신의 전략을 카운터치는 방향으로만 학습이 이루어져 최적의 정책으로 수렴하지 않고 그 주변에서 진동할 수도 있다는 문제점이 있다. 따라서 우리 팀은 opponent로 사용할 actor net 버전들로 구성된 pool을 만들어 놓고, 학습 데이터를 수집할 때 매 판마다 그 pool에서 랜덤한 opponent를 뽑아 현재 actor net이 그 opponent를 상대하도록 했다. 처음엔 파라미터가 초기화된 actor net 하나만 opponent pool에 들어있지만, 학습이 진행되어 pool에 있는 각 opponent에 대한 EMA(Exponential Moving Average) 승률 값 중 최소값이 60% 이상이 되면 현재 actor net의 checkpoint를 추가하게 하여 opponent pool의 크기가 점점 커지게 하였다. pool의 최대 크기는 10으로 설정하고, 크기가 10인 상태에서 opponent를 추가해야 될 때는 가장 예전에 추가됐던 opponent를 pop시켰다. 이에 더해 정책 업데이트가 계속 비슷한 곳을 빙글빙글 도는 것을 방지하기 위해 100 iteration마다 그 때의 actor net을 opponent pool에 넣어주었고, 이 opponent들은 절대 pop되지 않게 고정하였다. 또한 매 판 상대할 opponent를 뽑을 때는 그 확률이 EMA 승률에 반비례하도록, 즉 가장 어려워하는 opponent가 가장 자주 뽑히도록 하였다.

하지만 학습 초기 버전들을 중간평가에 제출하니 간간이 패배하는 게임이 몇 판 있었다. 그 판들을 분석해보니, 상대 팀이 굉장히 단순한 초반부 rush 전략이나 전사 5명씩 모아서

무작정 상대방 진영에 공격 보내는 전략을 사용할 때 어려워하는 것으로 보였다. 그래서 해당 리플레이 기록들을 보면서 상대방의 전략을 최대한 유사하게 플레이하는 두 휴리스틱 AI를 만들어 opponent pool에 처음부터 고정시켜 놓았다. 이 두 휴리스틱 AI들도 opponent를 뺏을 때 똑같이 EMA 승률 값에 따라 뺏히지만, opponent pool 크기가 10이 넘어가도 이 두 휴리스틱 ai들은 절대 pop되지 않도록 했다.

(6) 중간평가에서의 성능

PPO 알고리즘을 통한 self-play 데이터만으로 약 10시간 학습시킨 프로토타입 모델이 6월 30일 10시 중간평가에서 1등을 한 것을 확인한 뒤, 우리 팀은 observation space나 모델 구조, opponent pool 시스템을 더욱 정교화해 그 후 45시간(750 iteration) 동안 학습을 돌렸다. 이 강화학습 AI 1 버전은 7월 2일 10시부터 1등을 하기 시작해(최고 rating 점수는 3140) 7월 6일 21시에 2등으로 밀려나기 전까지 한 번도 중간평가에서 1등을 놓치지 않았을 정도로 예선 전반부에 독보적인 성능을 보였다.

3. 강화학습 AI 2 (2026.07.05.~2026.07.08.)

위에서 소개한 강화학습 AI 1 버전도 예선을 통과할만큼 충분히 강력했지만, 몇 가지 개선 아이디어들을 강화학습 AI 2 버전에 한꺼번에 적용한 뒤 학습시켜 예선 종료 전에 그 성능을 확인해보기로 했다.

(1) 상대방의 골드 수치 예측과 상대방 전사들의 이동 목표 예측

우선 이 게임을 하는 플레이어 입장에서 관측 불가능한 값이 2가지가 있다. 바로 상대방의 골드 수치와 상대방 전사들의 이동 목표인데, 강화학습 AI가 중간평가에서 대결할 때도 actor net은 이 2가지 정보를 정확하게 받지 못한다. 특히 강화학습 AI 1 버전에서는, 학습할 때는 상대방의 정확한 골드 수치를 actor net에 전달한 반면 제출용 코드에서는 상대방의 행동들만 근거로 해 예측한 골드 수치(실제 상대방의 골드 수치와는 차이가 꽤 컸다)를 넣어줘 distribution shift가 발생할 가능성이 컸다. 이동 중인 상대방 전사들의 목표 지점은 feature로 직접 넣어주진 않았기 때문에 distribution shift의 위험성은 덜했지만, 각 거점에 대해 상대방 전사들이 몇 턴 안에 올 수 있는지에 대한 feature만으로 상대방 전사들이 어느 기지를 타격하려 하는지를 학습할 수 있을지 미지수였다.

참고로 critic net은 어차피 PPO 학습 시에만 사용하기 때문에, 아군 뿐만 아니라 적군의 정확한 골드 수치와 적군 전사들의 정확한 이동 목표를 feature로 받도록 하였다.

(2) Auxiliary Target

상대방의 골드 수치와 상대방 전사들의 이동 목표, 이 두 문제를 완화하기 위해 auxiliary target을 설정하였다. 즉, PPO 학습시에 actor net이 "다음턴 상대방의 골드 수치"와 "다음턴 각 거점별로 상대방 전사들이 몇 턴 안에 올 수 있는지"를 예측하도록 해 그 오차도 learning signal로 사용한 것이다. 이렇게 하면 자동으로 actor net이 상대방의 골드 수치와 상대방 전사들의 움직임을 예측하는 데에 필요한 feature들에 주목하게 되기 때문에, 위 문제를 어느정도 완화할 수 있으리라 생각했다. 그리고 제출 코드와 똑같이 PPO 학습시에도 상대의 행동들로 예측한 상대의 골드 수치를 넣어 distribution shift가 발생하지 않도록

했다. 대신 이전 턴에 actor net이 예측한 "다음턴 상대방의 골드 수치" 값을 현재 턴의 actor net에 global feature로 추가해, 상대방의 행동들을 근거로 예측한 상대 골드 값이 실제 골드 값과 큰 차이가 날 때의 피해를 최소화하였다.

(3) 시뮬레이션을 통한 action 선택 정교화

제출환경에서는 각 에이전트에게 생각할 시간으로 한 턴 당 0.1초의 시간이 주어졌지만, 우리 팀의 actor net은 CPU 1코어 환경에서 1번 추론하는데에 약 20ms 정도면 충분하다는 테스트 결과가 나왔다. 게다가 여러 state들에 대한 observation 값을 하나의 batch로 묶어 한 번에 추론을 하면 0.1초 안에 수십번 추론하는 것도 가능해 보였다. 그래서 현재 상황에서 가장 유력한 몇 개의 action들을 뽑아 각 action을 했을 때의 결과를 시뮬레이션을 통해 예측하고, 가장 결과가 좋을 것으로 보이는 action을 선택하도록 하는 방식도 실험하였다.

시뮬레이션 방식에는 다양한 방식이 있을 수 있는데, 예를 들면 현재 상태(root)에서 내 입장에서 action 3개, 상대 입장에서 action 3개를 뽑아 $3 \times 3 = 9$ 개의 가짓수들을 시간이 허락할 때까지 시뮬레이션을 돌려서(내 입장, 상대 입장에서 action 1개씩만 뽑아서 게임 진행) 그 결과로 나온 게임 상태들에 대해 critic net으로 평가하는 방법이 있다.

그러나 3가지 정도의 서로 다른 시뮬레이션 방식들을 사용해본 결과, 0.1초 안에 각 action들의 가치를 정확히 평가하는 것은 불가능하다는 결론이 났다. 시뮬레이션을 적용한 에이전트가, 그냥 actor net의 출력값이 가장 높은 action만 항상 선택하는 에이전트보다 유의미하게 뛰어난 성능을 보이지 않았기 때문에, 최종 제출은 후자로 선택했다.

(4) 그 밖의 개선점들

(a) 맵 크기 feature 추가

강화학습 AI 1 버전을 중간평가에 제출했을 때 맵 크기가 상대적으로 작은 맵에서 종종 패배하는 모습을 보였다. 리플레이 분석을 해보니 맵 크기가 작을 때에는 원점, 즉 맵 정중앙에 있는 거점을 초반에 먼저 장악하는 것이 중요한데, 가운데로 빨리 전사를 보내지 않고 큰 맵에서 플레이 할 때처럼 그 주변의 거점들을 먼저 먹는 모습을 가끔씩 보였고, 이것이 결정적인 패배 요인일 것이라 생각했다. 따라서 global feature에 전체 맵의 절대적인 크기(x좌표 길이, y좌표 길이)를 추가하였다.

(b) 오프닝 한정 전사 분리

게임 처음에는 플레이어 당 본부 1개와 그 본부에 주둔해있는 전사 3명이 주어지고 시작한다. 따라서 첫 턴에 가장 좋은 플레이는, 3명의 전사들 중 1명은 본부에 남겨놓고, 남은 2명은 갈라져서 근처 거점 2곳으로 간 뒤 그곳에 기지 2개를 건설하고 주둔해있는 것이다. 하지만 우리 팀의 action space는 한 거점에 있는 전사들을 분리시키는 것이 불가능하기 때문에, 첫 턴에 전사 2명을 한꺼번에 근처의 빈 거점 한 곳으로 보낸 뒤, 그곳에 기지를 건설하고 그 다음 빈 거점으로 보내도록 학습됐다. 이는 빈 거점 두 곳이 너무 멀리 떨어져 있을 경우 상대보다 골드 수급이 너무 늦어지는 원인이 될 수 있기 때문에, 첫 턴에 한정해 actor net을 2번 추론시켜 한 거점에 이동 명령을 2개 내릴 수 있도록 했다. 첫 번째 추론으로 결정된 이동 명령은 전사 단 1명에게만 전달되며, 두 번째 추론으로 결정된 이동 명령은 남은 전사 1명에게 전달된다.

(5) 강화학습 AI 1 버전과 강화학습 AI 2 버전의 학습속도 차이

위 개선점들 덕분에 ‘강화학습 AI 2’ 버전이 ‘강화학습 AI 1’ 버전에 비해 실제로 학습이 가속화됐는지 확인하기 위해, opponent pool에 처음부터 고정시킨 두 휴리스틱 AI들에 대한 win rate EMA 그래프를 비교해보았다. 아래 그래프에서 rusher는 초반부 rush 전략을 구현한 휴리스틱 AI이고, japper는 전사 5명씩 모아서 무작정 상대방 진영에 공격 보내는 전략을 구현한 휴리스틱 AI이다. 두 강화학습 버전에 사용된 두 휴리스틱 AI 코드는 완전히 동일했기 때문에 객관적인 성능 비교가 가능하다. 또한 강화학습을 사용한 두 버전 모두 개선점들을 제외한 다른 hyperparameter들은 같은 값을 사용했다.



위 두 그래프를 보면 학습 초반에 rusher에 대한 승률도 버전 2가 버전 1보다 더 안정적으로 나오는 것을 볼 수 있고, japper에 대한 승률도 버전 2가 버전 1보다 훨씬 빠르게 100%로 수렴하는 것을 볼 수 있다. 따라서, 위에서 소개한 우리 팀의 개선점들이 학습 가속화에 유의미한 효과가 있었다고 할 수 있겠다.

(6) 중간평가에서의 성능

7월 6일 21시의 중간평가에서 강화학습 AI 1버전은 2등으로 밀려났지만, 7월 7일 15시의 중간평가에 강화학습 AI 2버전을 제출하자 다시 1등으로 올라오는 모습을 볼 수 있었다. 그러나 7월 7일 21시의 중간평가에서는 3등으로 다시 밀려났다. 그 이유를 추측해보면, 진 게임들을 분석해보니 상대방이 모두 특정 기지가 공격받았을 때 근처 기지에서 노동 중인 전사도 공격받은 기지를 방어하는 데에 사용하는 에이전트들이었다. 즉, 우리 팀 에이전트의 action space에서는 기지에서 노동 중인 전사는 이동이 불가능하기 때문에, 자가학습을 했던 우리 팀의 actor net은 이 정도라면 충분히 공격을 해서 상대방의 기지를 파괴할 수 있겠다고 생각한 상황도 상대방이 근처 기지에서 노동 중인 전사들도 이동시켜서 방어를 하게 되면 간단히 막혀버린 것이다. 게다가 강화학습 AI 2 버전은 PPO 자가학습 때 상대 전사들의 움직임을 예측하는 auxiliary target을 추가했기 때문에, 상대방이 기지에서 노동 중인 전사까지 이동시키지는 않을 것이라는 믿음이 더욱 컸을 것이다.

위 문제를 해결하기 위해 PPO 자가학습을 할 때 20%의 게임들에서는 opponent가 매 턴 10%의 확률로 기지에서 노동 중인 전사까지 이동시키도록 하는 noise를 추가하였다. 그 결과 월 8일 21시 마지막 중간평가에서 다시 강화학습 AI 2 버전으로 1등을 하였으며, 그대로 최종평가에 제출하였다.

4. 부록

(1) Hyperparameter List

강화학습 AI 1, 2 두 버전 모두 학습시킬 때 1~250 iteration은 phase 1, 251~500

iteration은 phase 2, 501~750 iteration은 phase 3으로 나눠서 phase마다 hyperparameter를 조금씩 다르게 학습시켰다.

Phase 1	Phase 2	Phase 3
동시에 진행되는 게임 수: 256 iteration마다 수집하는 데이터: 200000 step gamma: 0.997 GAE lambda: 0.95 PPO clip range: 0.2 lr: 0.0005 epochs: 5 minibatch: 4096 entropy coefficient: 0.005 auxiliary loss coefficient: 0.15 maximum gradadient norm: 1.0 d_model(트랜스포머 width): 64	동시에 진행되는 게임 수: 256 iteration마다 수집하는 데이터: 250000 step gamma: 0.997 GAE lambda: 0.95 PPO clip range: 0.2 lr: 0.0002 epochs: 4 minibatch: 4096 entropy coefficient: 0.002 auxiliary loss coefficient: 0.15 maximum gradadient norm: 1.0 d_model(트랜스포머 width): 64	동시에 진행되는 게임 수: 256 iteration마다 수집하는 데이터: 300000 step gamma: 0.997 GAE lambda: 0.95 PPO clip range: 0.2 lr: 0.0001 epochs: 3 minibatch: 4096 entropy coefficient: 0.001 auxiliary loss coefficient: 0.15 maximum gradadient norm: 1.0 d_model(트랜스포머 width): 64

(2) 학습에 사용한 컴퓨팅 자원

우리 팀은 강화학습에 데스크탑 1대만을 사용하였다.

CPU: 13th Gen Intel(R) Core(TM) i5-13600K(3.50 GHz)

RAM: 32.0GB

GPU: NVIDIA GeForce RTX 4080 (16 GB)

(3) 남은 개선 아이디어들

(a) MCTS(Monte-Carlo Tree Search)와 같은 model-based RL: 턴제 게임이고, 매 턴 계산해야 하는 양이 그렇게 많지 않아 알파제로의 방식처럼 MCTS를 RL에 사용할 수 있을 것 같았다. 하지만 action space가 꽤 복잡하고 완전 정보 게임이 아닌데다가, 우리 팀이 사용할 수 있는 컴퓨팅 자원이 그렇게 많은 편이 아니라 포기하였다.

(b) 정교한 opponent league 시스템 운영: 우리 팀의 opponent pool은 간단한 휴리스틱 AI 2개, 가장 최근의 actor net 10개, 그리고 100 iteration마다 저장한 actor net들로 이루어졌다. 그러나 AlphaStar 논문을 보면, 현재 학습 중인 main actor net에 대한 카운터 전략만을 학습하는 main exploiter, 전체 opponent pool의 카운터 전략만을 학습하는 league exploiter와 같이 main actor net 말고도 여러 종류의 actor net들을 학습해 나가면서 정책 수렴을 더욱 안정화시킬 수 있다. 그러나 우리 팀이 사용할 수 있는 컴퓨팅 자원과 예선 시간의 한계 때문에 시도해보지는 못했다.

(c) RNN 방식 사용: 우리 팀이 사용한 actor net과 critic net의 구조를 보면 거의 트랜스포머 모델로만 이루어져 있다. 여기에 RNN 방식을 접목시켜 현재 턴에서 이전 턴들의 정보도 고려할 수 있게 했다면, 상대의 전체적인 전략에 따라 자신의 전략을 유기적으로 변화시키는 AI도 학습시킬 수 있었을지 모른다. 그러나 시간이 그렇게 충분하지 않은 상황에서 이러한 도전적인 아이디어를 실험하는 것은 오히려 코드의 복잡성을 증가시키고 학습의 안정성을 해칠 위험이 크다고 생각해 시도해보지는 않았다.

(d) 기지에서 노동 중인 전사도 전투에 동원하는 action space 설계: 강화학습 AI 2 버전의 중간평가에서의 성능에서 설명했다시피, 우리 팀은 이동명령의 action space를 (source 거점, target거점)의 pair로 축소시킴으로써 빠른 학습 속도를 얻어냈지만, 그 대가로 한 거

점에 있는 전사들을 스플릿하거나 이미 기지에서 노동 중인 전사를 능동적인 전투 자원으로 활용하는 정책은 학습시키지 못했다. 그 결과 자가학습만으로 학습을 완료한 우리 팀의 AI는 모든 상대방들도 자신처럼 플레이할 것이라 생각했고, 특히 기지에서 노동 중인 전사까지 다른 기지의 수비에 사용하는 "0262", "역대급 물로켓" 등 팀들의 AI에 대해 비교적 약한 모습을 보였다. 만약 기지에서 노동 중인 전사도 전투에 동원할 수 있는 action space를 설계한다면, 학습 시간은 좀 더 늘어나겠지만 이 AI들의 전략도 충분히 대응할 수 있는 AI가 만들어졌을 것이다. 하지만 마찬가지로 시간과 컴퓨팅 자원이 부족해 시도해보지는 못했다.

예선 후기

- 우리 팀은 내부 테스트나 중간평가에서의 리플레이들을 참고하면서 대회를 어떻게 진행해나갈지에 대한 큰 방향성을 결정하고, 그 결정 내용에 따라 LLM에게 프롬프트를 입력하고 코드를 작성하게 하는 식으로 LLM을 활용했다. 특히 claude code의 Opus 4.8 모델을 많이 사용하였는데, 강화학습 관련 코드는 대체적으로 실수 없이 잘 작성해준 것 같다. 그러나 휴리스틱 AI를 claude code로 만들 때는 애를 좀 먹었는데, 이 정도면 우리 팀이 원하는 휴리스틱 AI의 전략이 뭔지 구체적으로 설명했다 싶어도 그대로 행동하지 않는 코너 케이스들이 많은 코드를 작성해주었다. 그래서 그 뒤로 몇 번이나 수정 사항을 알려줘야 했고, 이 때문에 많은 토큰들을 낭비했다. 아무래도 휴리스틱 AI 플레이어가 어떻게 플레이하기를 바란다고 프롬프트에 적기보다, 어떤 정보를 저장하고 있어야 하고 어떤 알고리즘적인 원리로 그렇게 플레이하기를 바란다고까지 적어줘야 훨씬 실수 없이 깔끔하게 코드를 구성해주는 듯하다. 예선 막바지에 claude의 Fable 5 모델도 사용해봤는데 Opus 4.8보다 훨씬 간결하면서도 원하는 대로 잘 작동하는 휴리스틱 AI를 만들어줬다.

- 이번 NYPC 예선 직전에 "Orbit Wars"라는 kaggle 대회(<https://www.kaggle.com/competitions/orbit-wars/overview>)에 참가했는데, 좋은 성적을 거두진 못했지만 이때 얻었던 교훈 덕에 이번 NYPC 예선에서 좋은 성능을 내는 강화학습 AI를 빠르게 만들 수 있었다.

- 전반적인 대회 진행에 대해서는 큰 불만 없이 만족스럽게 예선을 마칠 수 있었다. 그래도 만약 가능하다면 대회 시간을 좀 더 늘려주면 좋겠다. 사실 10일 정도의 시간으로는 좋은 강화학습 AI를 만들기 어렵다(최신 GPU가 탑재된 가정용 데스크탑에서 강화학습 1번에 2~3일 정도 걸린다고 가정하면, 한 팀에서 정말 많아야 5번 정도의 실험을 할 수 있는 기간이다). 작년 본선은 AI 개발 시간으로 6시간이 주어졌다고 알고 있는데, 이 정도 시간이면 강화학습과 같이 신경망 모델을 사용하는 방법은 아예 불가능하다. 어느 팀이 휴리스틱 AI를 더 효율적으로, 정교하게 짰는지 평가하는 대회보다는, 시간을 더 늘려서(1박 2일 정도로) 훨씬 더 다양한 방법들을 탐색할 수 있도록 돕는 대회가 더 좋은 대회가 아닐까하는 생각이 들었다.